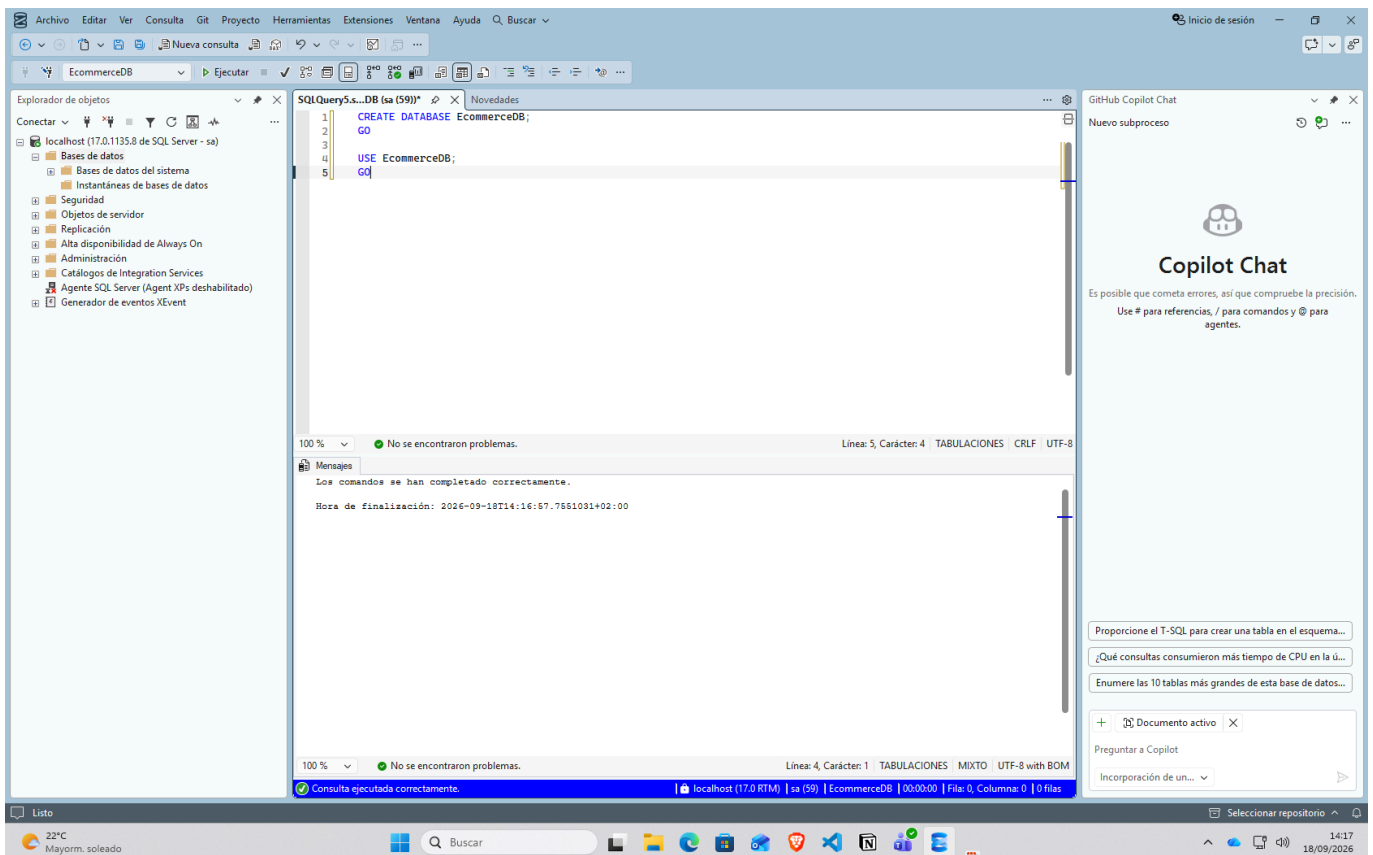


Paso 1. Crear la base de datos

En este paso crearemos una base de datos llamada 'EcommerceDB'

```
CREATE DATABASE EcommerceDB;  
GO  
  
USE EcommerceDB;  
GO
```



Paso 2. Crear las tablas

En este paso crearemos 3 tablas Supplier, Category, Product

```
USE EcommerceDB;  
GO  
  
-- Create Supplier table  
CREATE TABLE Supplier (  
    SupplierID INT PRIMARY KEY IDENTITY(1,1),  
    SupplierName NVARCHAR(100) NOT NULL UNIQUE,  
    Country NVARCHAR(50) NOT NULL,  
    Email NVARCHAR(100),
```

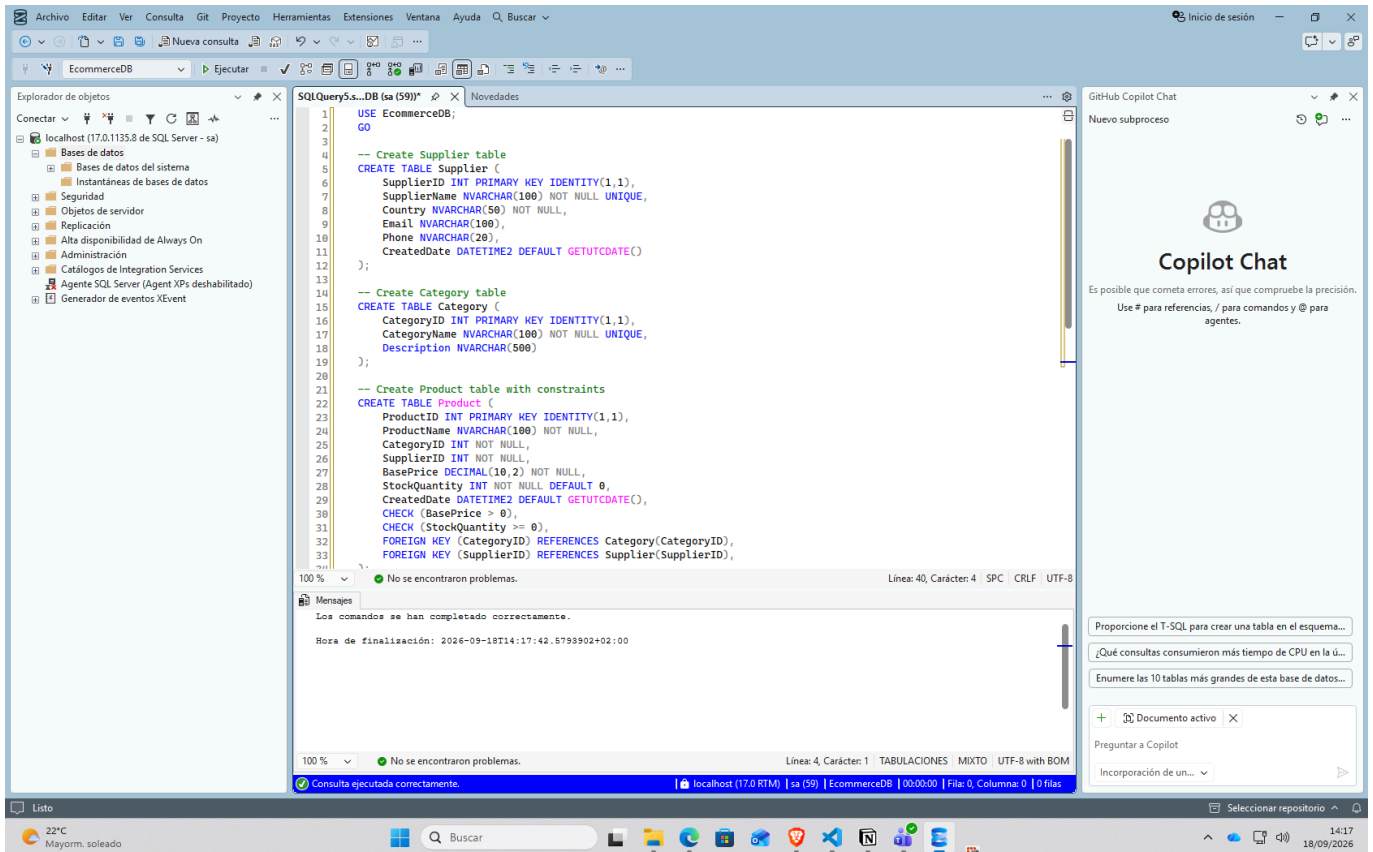
```
    Phone NVARCHAR(20),
    CreatedDate DATETIME2 DEFAULT GETUTCDATE()
);

-- Create Category table
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) NOT NULL UNIQUE,
    Description NVARCHAR(500)
);

-- Create Product table with constraints
CREATE TABLE Product (
    ProductID INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(100) NOT NULL,
    CategoryID INT NOT NULL,
    SupplierID INT NOT NULL,
    BasePrice DECIMAL(10,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    CreatedDate DATETIME2 DEFAULT GETUTCDATE(),
    CHECK (BasePrice > 0),
    CHECK (StockQuantity >= 0),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID),
);

-- Create indexes
CREATE INDEX IX_Category ON Product(CategoryID);
CREATE INDEX IX_Supplier ON Product(SupplierID);

GO
```



Paso 3. Insertar datos en las tablas

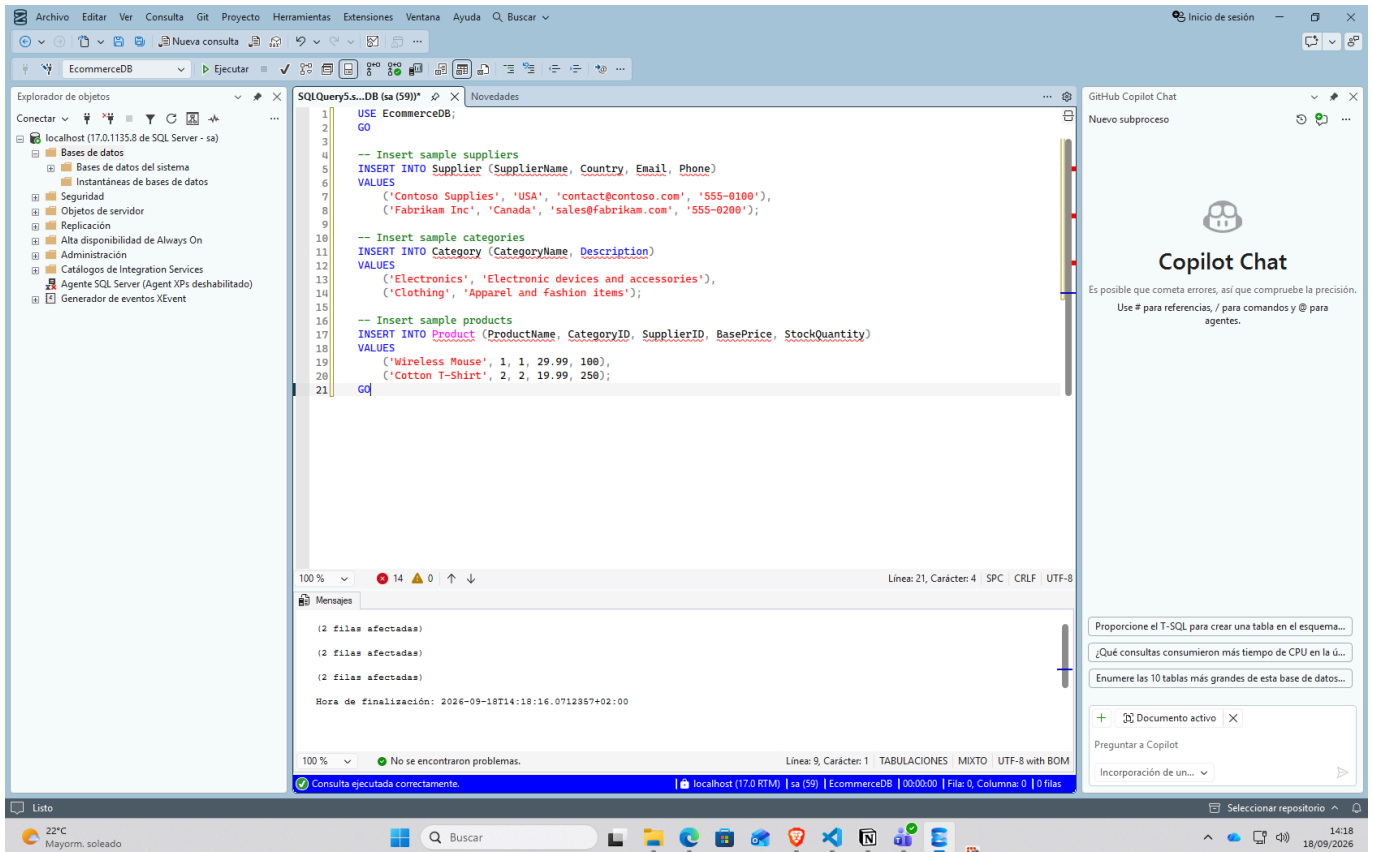
En este paso insertaremos datos de ejemplo en las tablas para trabajar con ellos.

```
USE EcommerceDB;
GO

-- Insert sample suppliers
INSERT INTO Supplier (SupplierName, Country, Email, Phone)
VALUES
    ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
    ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');

-- Insert sample categories
INSERT INTO Category (CategoryName, Description)
VALUES
    ('Electronics', 'Electronic devices and accessories'),
    ('Clothing', 'Apparel and fashion items');

-- Insert sample products
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice,
StockQuantity)
VALUES
    ('Wireless Mouse', 1, 1, 29.99, 100),
    ('Cotton T-Shirt', 2, 2, 19.99, 250);
GO
```



Paso 4. Crear una tabla temporal

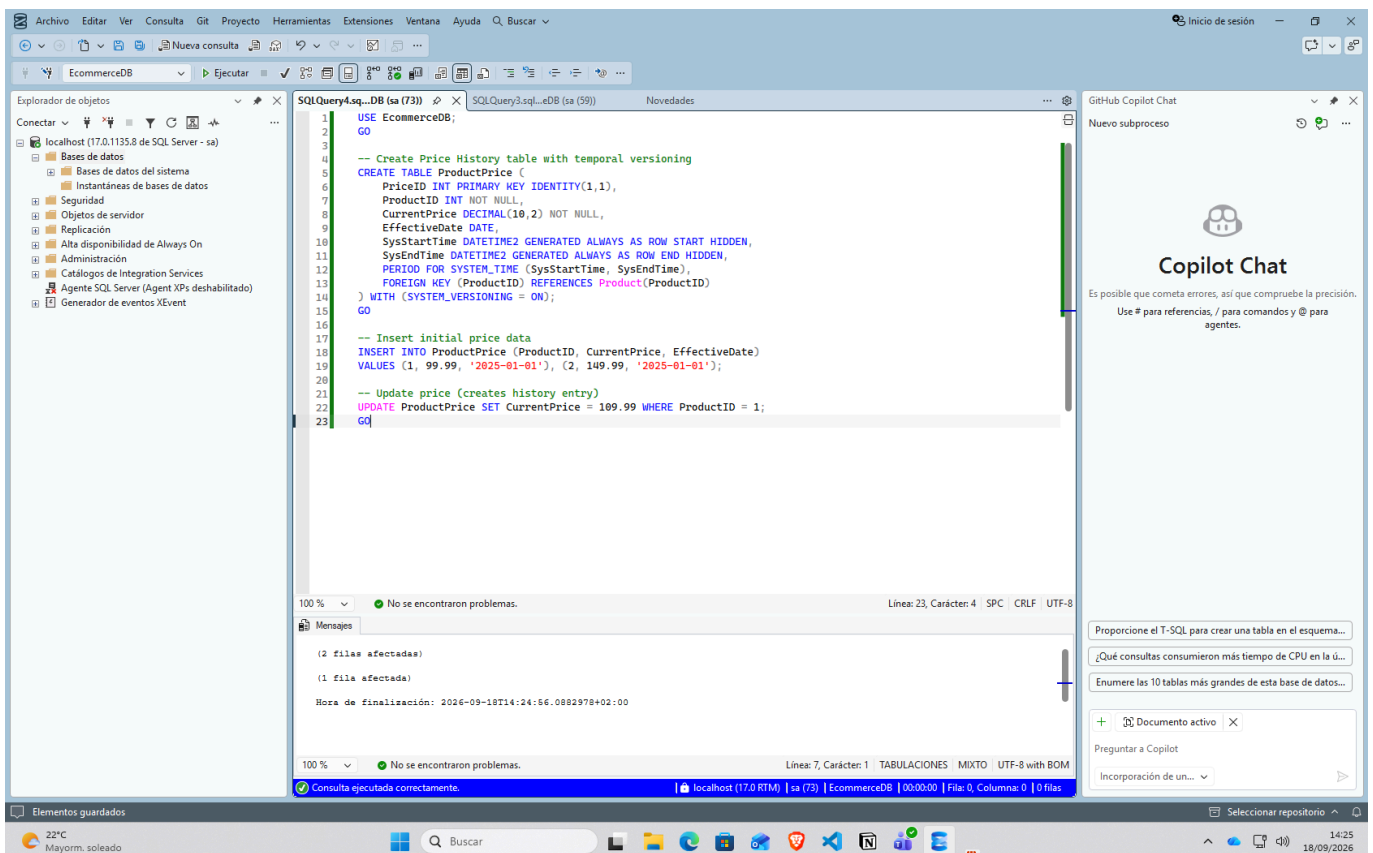
En este paso creamos una tabla temporal

```
USE EcommerceDB;
GO

-- Create Price History table with temporal versioning
CREATE TABLE ProductPrice (
    PriceID INT PRIMARY KEY IDENTITY(1,1),
    ProductID INT NOT NULL,
    CurrentPrice DECIMAL(10,2) NOT NULL,
    EffectiveDate DATE,
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
) WITH (SYSTEM_VERSIONING = ON);
GO

-- Insert initial price data
INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');

-- Update price (creates history entry)
UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1;
GO
```

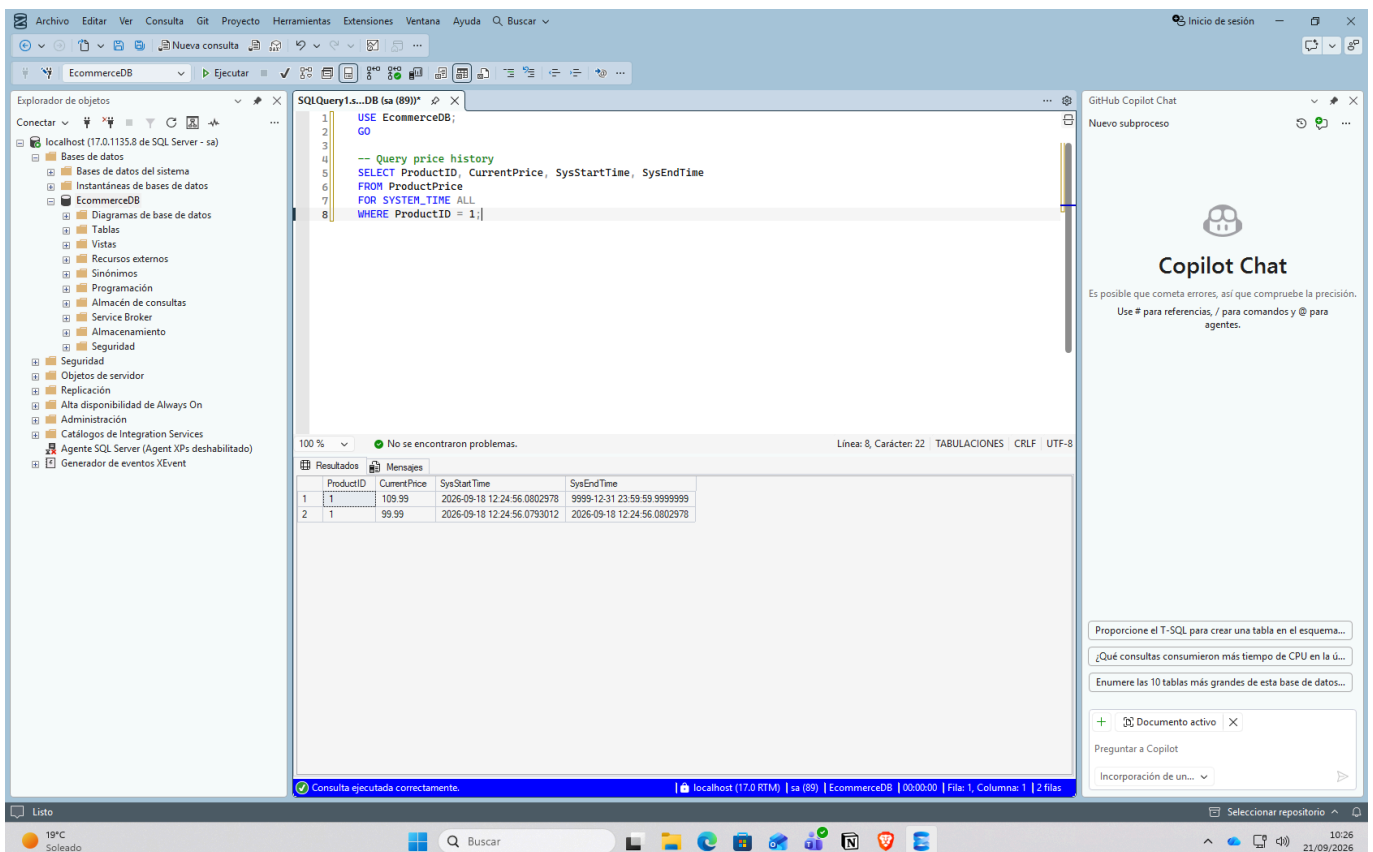


Paso 5. Consulta el historial de precios

En este paso realizamos una consulta para ver el historial de los precios

```
USE EcommerceDB;
GO

-- Query price history
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice
FOR SYSTEM_TIME ALL
WHERE ProductID = 1;
```



The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'EcommerceDB' database structure. The central pane shows a query window with the following SQL code:

```
USE EcommerceDB;
GO

-- Query price history
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice
FOR SYSTEM_TIME ALL
WHERE ProductID = 1;
```

The bottom pane displays the query results in a table with the following data:

ProductID	CurrentPrice	SysStartTime	SysEndTime
1	109.99	2026-09-18 12:24:56.0802978	9999-12-31 23:59:59.9999999
2	99.99	2026-09-18 12:24:56.0793012	2026-09-18 12:24:56.0802978

The status bar at the bottom indicates 'Consulta ejecutada correctamente.' (Query executed successfully).

Paso 6. Añadir metadata JSON

En este paso guardamos los metadatos en json para facilitar que las búsquedas por color.

```
USE EcommerceDB;
GO

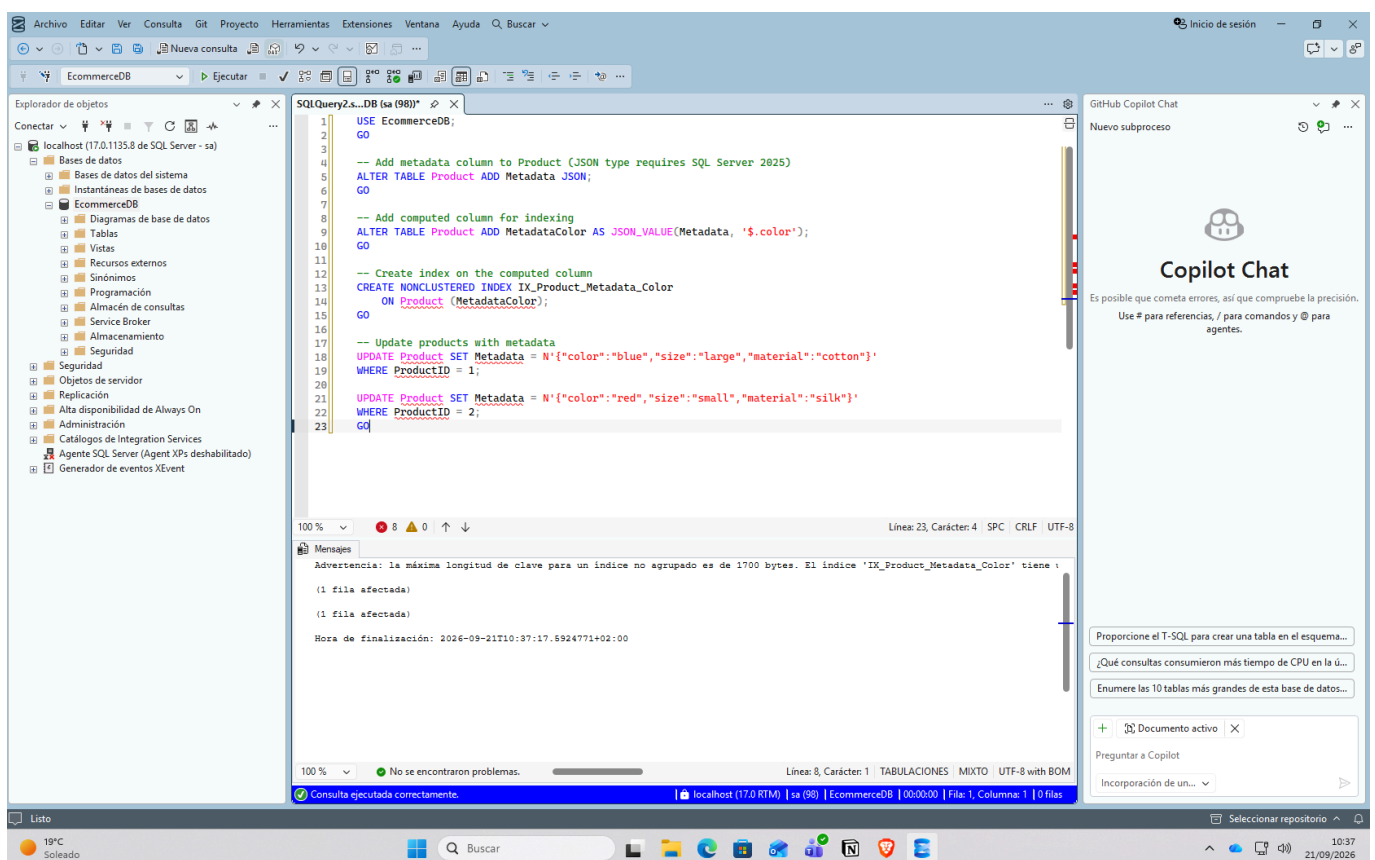
-- Add metadata column to Product (JSON type requires SQL Server 2025)
ALTER TABLE Product ADD Metadata JSON;
GO

-- Add computed column for indexing
ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
GO
```

```
-- Create index on the computed column
CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
ON Product (MetadataColor);
GO

-- Update products with metadata
UPDATE Product SET Metadata =
N'{"color":"blue","size":"large","material":"cotton"}'
WHERE ProductID = 1;

UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"silk"}'
WHERE ProductID = 2;
GO
```



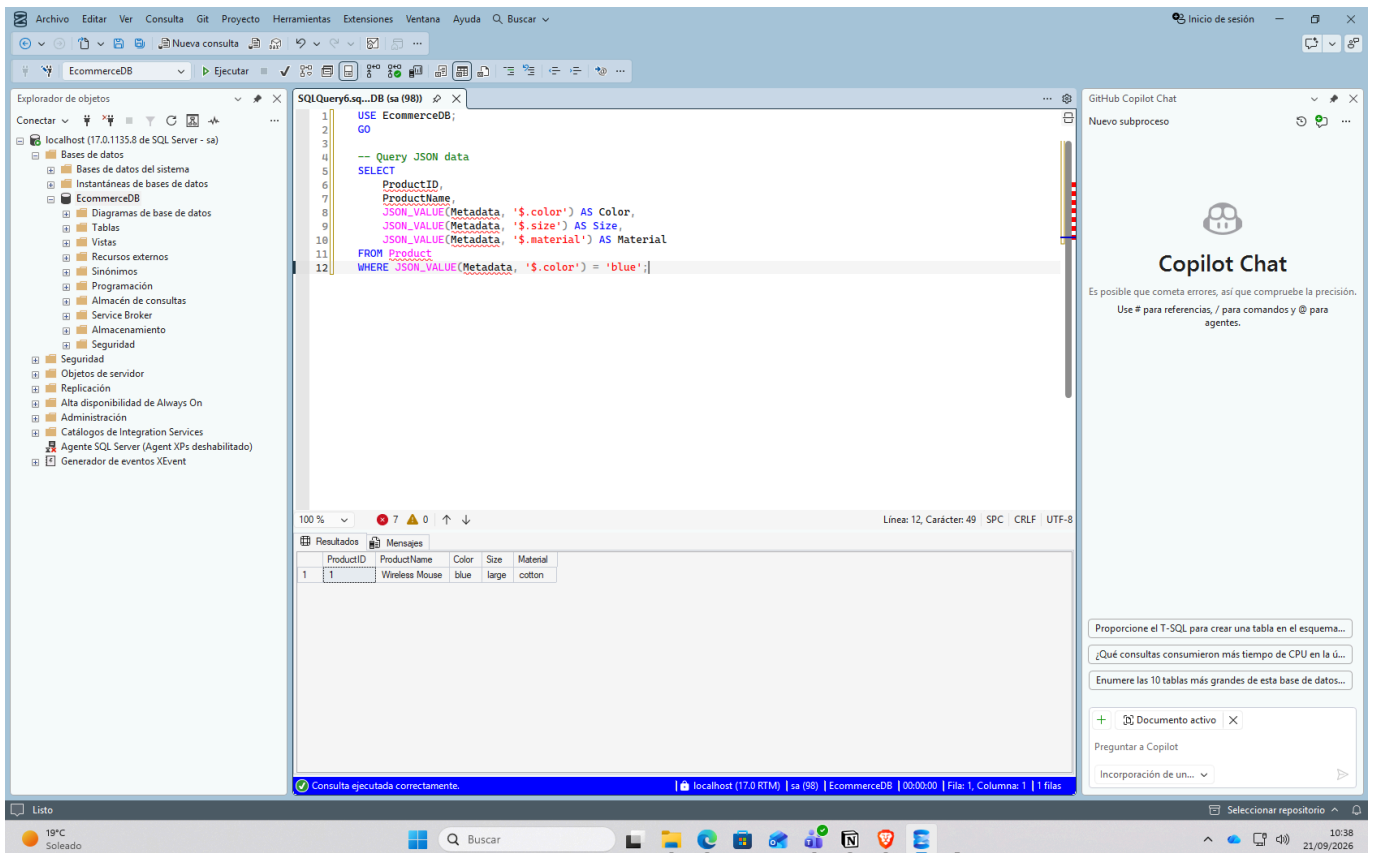
Paso 7. Consultar los datos JSON

En este paso probaremos si funcionan las consultas sobre los metadatos que añadimos en json.

```
USE EcommerceDB;
GO

-- Query JSON data
SELECT
    ProductID,
    ProductName,
    JSON_VALUE(Metadata, '$.color') AS Color,
```

```
JSON_VALUE(Metadata, '$.size') AS Size,
JSON_VALUE(Metadata, '$.material') AS Material
FROM Product
WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
```



Paso 8. Crear una tabla particionada

En este paso creamos particiones para poder trabajar solo con los datos que necesitamos.

```
USE EcommerceDB;
GO

-- Create partition function for order dates
-- Use RANGE RIGHT for date columns to keep same-day values together
CREATE PARTITION FUNCTION PF_OrderDate (DATE)
AS RANGE RIGHT FOR VALUES
  ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');

-- Create partition scheme (single filegroup recommended)
CREATE PARTITION SCHEME PS_OrderDate
AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);

-- Create partitioned Order table
-- Include OrderDate in primary key for clustered index alignment
CREATE TABLE [Order] (
  OrderID BIGINT IDENTITY(1,1),
  OrderDate DATE NOT NULL,
```

```

CustomerName NVARCHAR(100) NOT NULL,
TotalAmount DECIMAL(12,2) NOT NULL,
OrderStatus NVARCHAR(20) DEFAULT 'Pending',
CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
CHECK (TotalAmount > 0),
CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered',
'Cancelled'))
) ON PS_OrderDate(OrderDate);

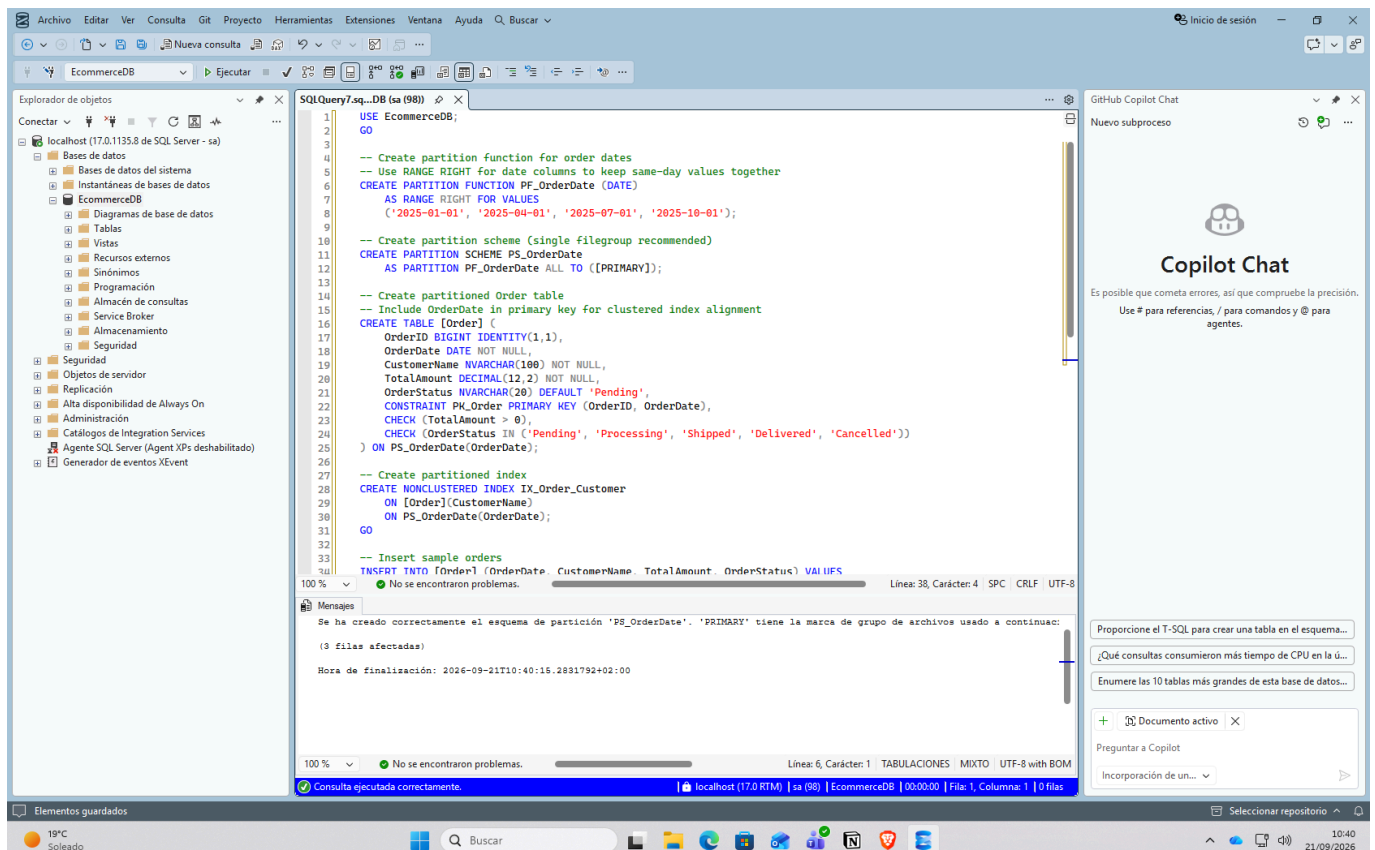
-- Create partitioned index
CREATE NONCLUSTERED INDEX IX_Order_Customer
ON [Order](CustomerName)
ON PS_OrderDate(OrderDate);

GO

-- Insert sample orders
INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
('2025-01-15', 'John Smith', 299.97, 'Delivered'),
('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
('2025-06-10', 'Bob Johnson', 449.95, 'Processing');

GO

```

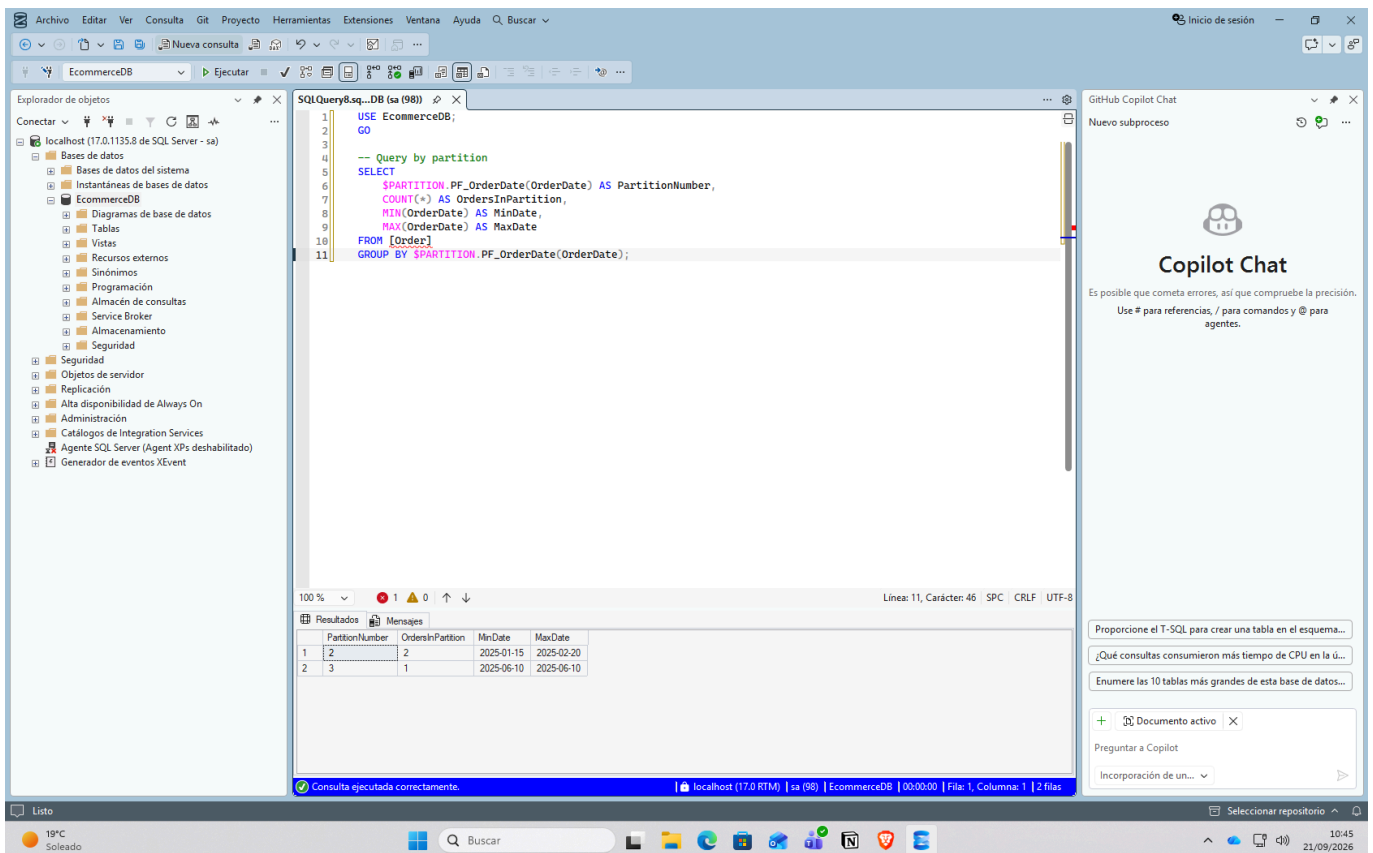


Paso 9. Consultar las particiones

En este paso consultamos en las particiones que hemos creado para determinar en qué partición se encuentra cada pedido.

```
USE EcommerceDB;
GO

-- Query by partition
SELECT
    $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
    COUNT(*) AS OrdersInPartition,
    MIN(OrderDate) AS MinDate,
    MAX(OrderDate) AS MaxDate
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```



SQLQuery8.sql...DB (sa (98))

```
1 USE EcommerceDB;
2 GO
3
4 -- Query by partition
5 SELECT
6     $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
7     COUNT(*) AS OrdersInPartition,
8     MIN(OrderDate) AS MinDate,
9     MAX(OrderDate) AS MaxDate
10 FROM [Order]
11 GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```

PartitionNumber	OrdersInPartition	MinDate	MaxDate
2	2	2025-01-15	2025-02-20
3	1	2025-06-10	2025-06-10

100 % 1 0 Linea: 11, Carácter: 46 SPC CRLF UTF-8

Resultados Mensajes

Consulta ejecutada correctamente. localhost (17.0 RTM) | sa (98) | EcommerceDB | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

GitHub Copilot Chat

Nuevo subproceso

Copilot Chat

Es posible que cometa errores, así que compruebe la precisión. Use # para referencias, / para comandos y @ para agentes.

Proporcione el T-SQL para crear una tabla en el esquema...

¿Qué consultas consumieron más tiempo de CPU en la ú...

Enumere las 10 tablas más grandes de esta base de datos...

+ Documento activo X

Preguntar a Copilot

Incorporación de un...

Paso 10. Crear una SEQUENCE y OrderDetail

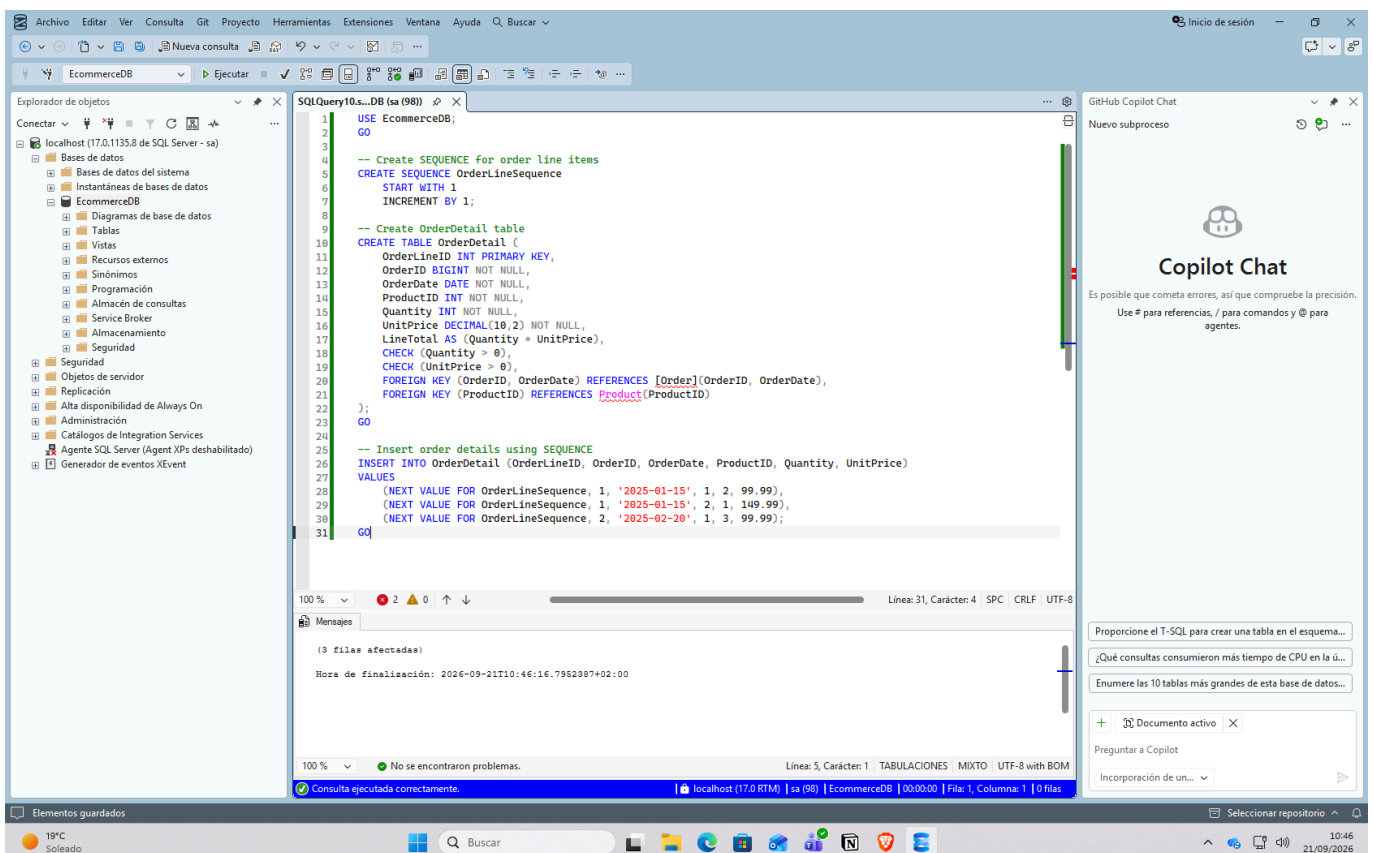
En este paso creamos una SEQUENCE para generar números de forma incremental, y luego OrderDetail que representa las líneas individuales de cada pedido.

```
USE EcommerceDB;
GO

-- Create SEQUENCE for order line items
CREATE SEQUENCE OrderLineSequence
    START WITH 1
    INCREMENT BY 1;
```

```
-- Create OrderDetail table
CREATE TABLE OrderDetail (
    OrderLineID INT PRIMARY KEY,
    OrderID BIGINT NOT NULL,
    OrderDate DATE NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    LineTotal AS (Quantity * UnitPrice),
    CHECK (Quantity > 0),
    CHECK (UnitPrice > 0),
    FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
GO

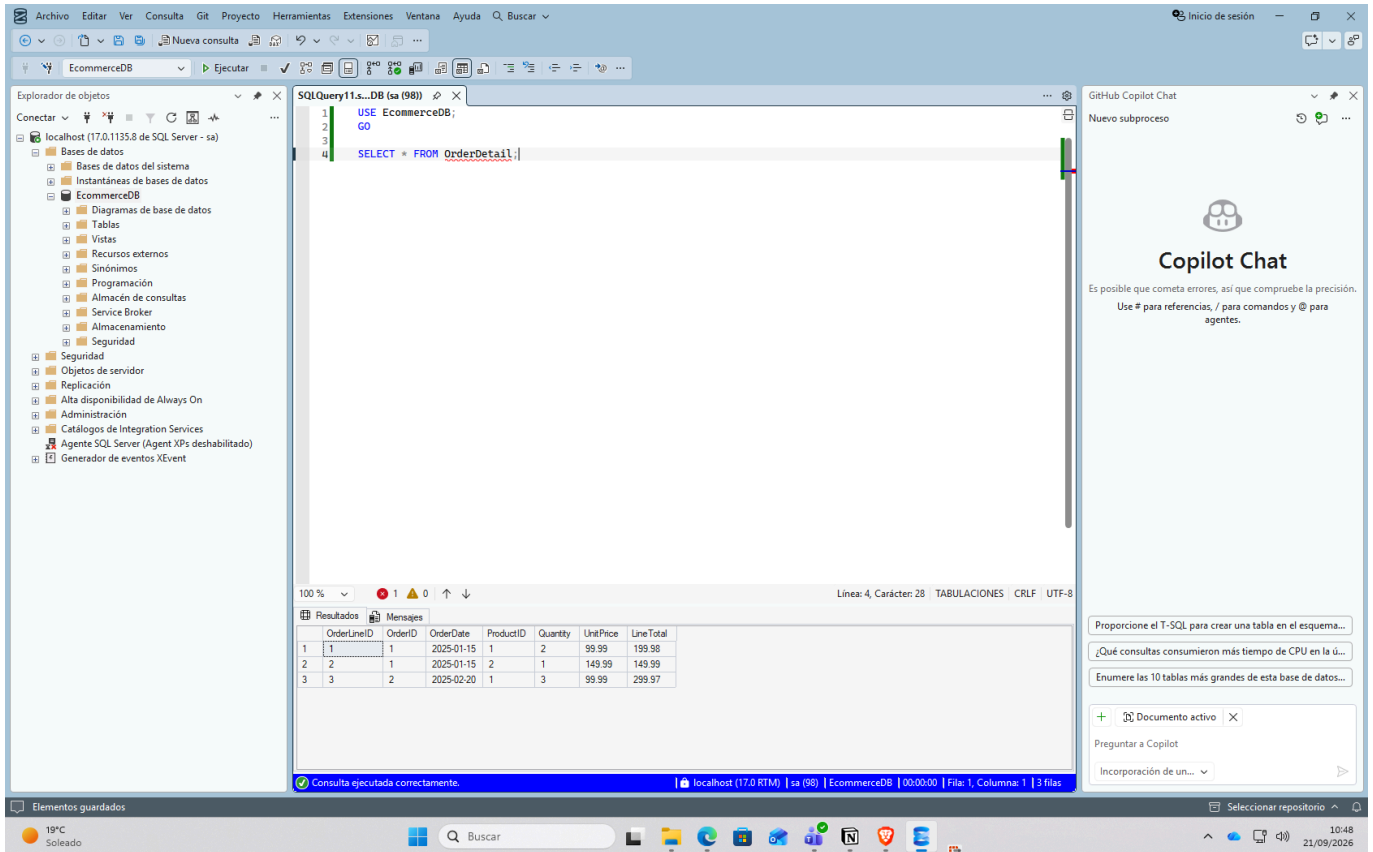
-- Insert order details using SEQUENCE
INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity,
UnitPrice)
VALUES
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
    (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);
GO
```



Paso 11. Comprobar OrderDetail

En este paso comprobamos los registros de OrderDetail

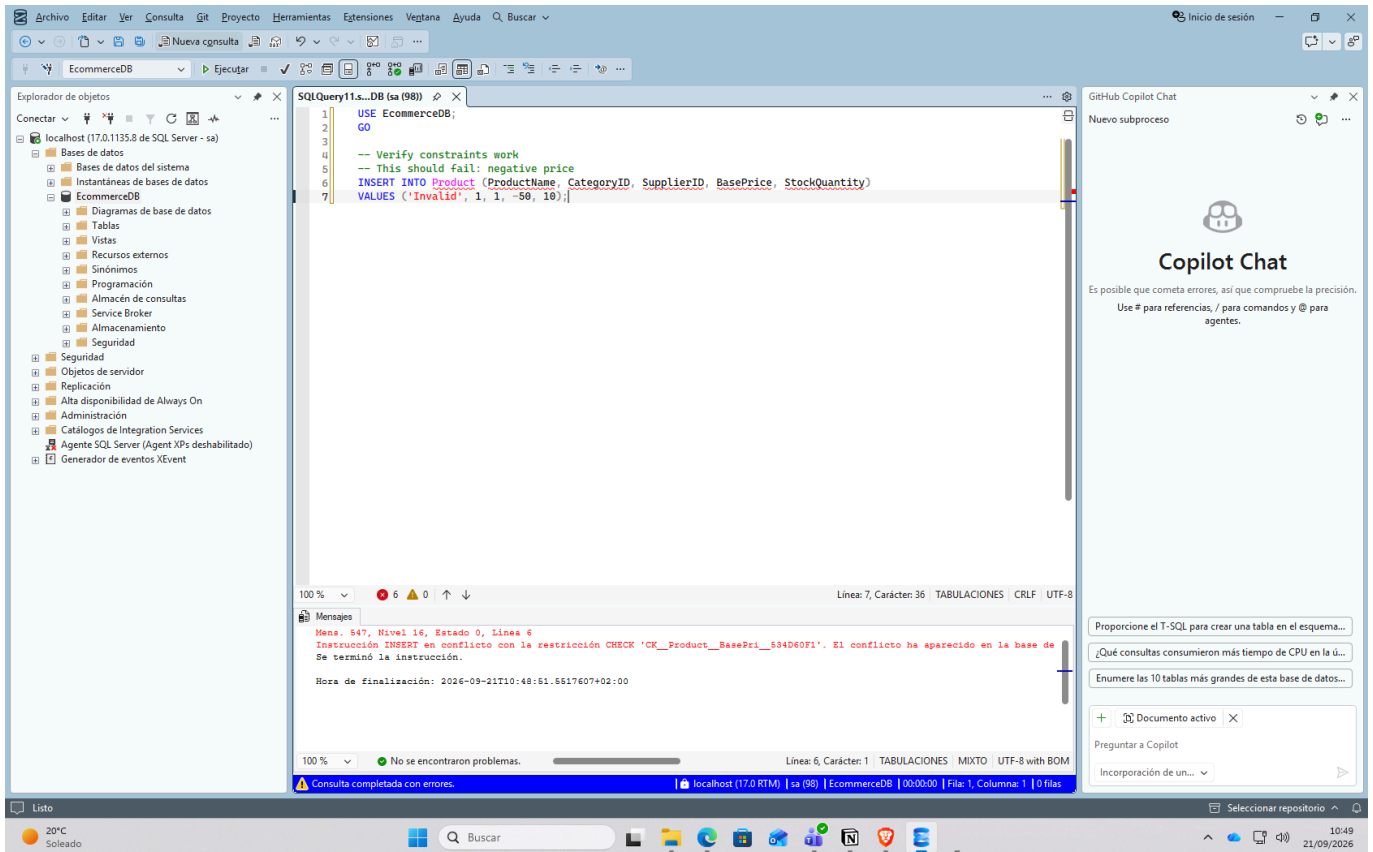
```
USE EcommerceDB;  
GO  
  
SELECT * FROM OrderDetail;
```



Paso 12. Comprobar que funcionan los constraints

En este paso se comprobará si el constraint funciona.

```
USE EcommerceDB;  
GO  
  
-- Verify constraints work  
-- This should fail: negative price  
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice,  
StockQuantity)  
VALUES ('Invalid', 1, 1, -50, 10);
```



Paso 13. Verificación final

En este ultimo paso se verificará el json, el particionamiento y la tabla temporal.

```
USE EcommerceDB;
GO

-- Verify JSON queries work
SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
FROM Product
WHERE Metadata IS NOT NULL;

-- Verify partitioning
SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);

-- Verify temporal table
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice FOR SYSTEM_TIME ALL
ORDER BY ProductID, SysStartTime;
```

ArchivoEditarVerConsultaGitProyectoHerramientasExtensionesVentanaAyudaQ Buscar

Nueva consulta

Ejecutar

Ver

Guardar

Cancelar

Repetir

Detener

Actualizar

Eliminar

Compartir

Imprimir

Revertir

Reparar

Reemplazar

Comentarios

Formato

Formato de texto

Formato de código

Formato de SQL

Formato de JSON

Formato de XML

Formato de YAML

Formato de Markdown

Formato de HTML

Formato de CSS

Formato de JavaScript

Formato de Python

Formato de Java

Formato de C#

Formato de PHP

Formato de Ruby

Formato de Go

Formato de Rust

Formato de Swift

Formato de Kotlin

Formato de Scala

Formato de F#

Formato de VB.NET

Formato de C++

Formato de C

Formato de Fortran

Formato de MATLAB

Formato de Octave

Formato de Julia

Formato de R

Formato de Perl

Formato de Python

Formato de Java

Formato de C#

Formato de PHP

Formato de Ruby

Formato de Go

Formato de Rust

Formato de Swift

Formato de Kotlin

Formato de Scala

Formato de F#

Formato de VB.NET

Formato de C++

Formato de C

Formato de Fortran

Formato de MATLAB

Formato de Octave

Formato de Julia

Formato de R

Formato de Perl

Explorador de objetos

Conectar

localhost (17.0.1135.8 de SQL Server - sa)

Bases de datos

Bases de datos del sistema

Instantáneas de bases de datos

EcommerceDB

Diagramas de base de datos

Tablas

Vistas

Recursos externos

Sinónimos

Programación

Almacén de consultas

Service Broker

Almacenamiento

Seguridad

Objetos de servidor

Replicación

Alta disponibilidad de Always On

Administración

Catálogos de Integration Services

Agente SQL Server (Agent XPs deshabilitado)

Generador de eventos XEvent

SQLQuery12.s...DB (sa (98))

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

100 %

12

0

↑

↓

Línea: 17, Carácter: 35

TABULACIONES

CRLF

UTF-8

Resultados

Messages

ProductID

CurrentPrice

SysStartTime

SysEndTime

1

2

2026-09-21 08:34:57.8526408

9999-12-31 23:59:59.9999999

2

149.99

2026-09-21 08:34:57.8526408

9999-12-31 23:59:59.9999999

Consultas ejecutadas correctamente.

localhost (17.0 RTM) | sa (98) | EcommerceDB | 00:00:00 | Fila: 1, Columna: 1 | 6 filas

Elementos guardados

20°C

Soleado

Buscar

Seleccionar repositorio

10:50

21/09/2026

GitHub Copilot Chat

Nuevo subproceso

Copilot Chat

Es posible que cometa errores, así que compruebe la precisión.

Use # para referencias, / para comandos y @ para agentes.

Proporcione el T-SQL para crear una tabla en el esquema...

¿Qué consultas consumieron más tiempo de CPU en la ú...

Enumere las 10 tablas más grandes de esta base de datos...

+ Documento activo X

Preguntar a Copilot

Incorporación de un...